

Filtro de Dados e Compressão para Computação em Névoa em IoT na Agricultura Inteligente utilizando Rust e WebAssembly

Victor Cobo Figueiro

Curso de Bacharelado em Ciência da Computação
Universidade Federal do ABC - UFABC

Julho de 2026

Orientador: Prof. Dr. Carlos Alberto Kamienski



- 1 Introdução e Objetivos
- 2 Referencial Teórico
- 3 Arquitetura e Implementação
- 4 Metodologia Experimental
- 5 Resultados e Discussões
- 6 Conclusões

- **Agricultura Inteligente (Smart Agriculture):**

- Sensoriamento contínuo de variáveis ambientais (temperatura, umidade, solo).
- Auxilia na otimização de recursos (irrigação, defensivos) e prevenção de pragas.

- **Gargalo de Rede em Ambientes Remotos:**

- Links de comunicação de longo alcance e baixo consumo (*LoRaWAN*, *satélite*).
- Banda passante extremamente limitada e alto custo energético por byte transmitido.
- Envio direto de dados brutos à nuvem causa congestionamento e morte prematura das baterias dos sensores.

O Problema da Telemetria IoT

O Paradoxo do Sensoriamento Físico

Sensores de temperatura e umidade emitem leituras decimais redundantes de alta frequência. Enviar continuamente dados flutuantes decimais (ex: 24.53, 60.12) satura canais de rádio de baixa taxa sem agregar valor agrônômico direto instantâneo.

Desafios:

- ❶ Reduzir o volume de dados na ponta da rede (Thing e Névoa).
- ❷ Manter a relevância agrônômica das informações transmitidas.
- ❸ Viabilizar uma arquitetura de orquestração leve no contínuo IoT-Nuvem.

- **Geral:** Propor, implementar e avaliar um ecossistema distribuído de computação em névoa baseado em **Rust** e **WebAssembly (WASM/WASI)** para filtragem e compressão de dados na borda.
- **Específicos:**
 - ➊ Adaptar o classificador k -Vizinhos Mais Próximos (k NN, com $k = 3$) para converter medidas físicas de temperatura e umidade em status de saúde do tomateiro de caractere único.
 - ➋ Avaliar a compressão temporal via codificação RLE (*Run Length Encoding*).
 - ➌ Projetar um simulador estocástico off-line (KDE + Box-Muller) para validar estatisticamente a robustez do ecossistema.
 - ➍ Medir a eficácia e o consumo de rede comparativo das diferentes configurações de buffer.

- **Computação em Névoa (Fog Computing):**

- Camada intermediária que descentraliza a computação da Nuvem para locais próximos aos sensores.
- Mitiga latência, economiza banda passante de uplink e viabiliza resposta ágil no campo.

- **O Cloud Continuum (IoTinuum):**

- Orquestração unificada de cargas lógicas que trafegam entre as camadas de *Things*, *Edge/Fog* e *Cloud*.
- **Problema de Orquestradores Clássicos (Docker/K8s/KubeEdge):** Pegada de memória e CPU muito pesada para nós de borda fracos (gateways agrícolas de baixo custo).

- **WebAssembly (WASM):**

- Formato de instrução binária de baixo nível executado em máquina virtual baseada em pilha com isolamento estrito (*sandboxing*).
- *WebAssembly System Interface* (WASI) padroniza o acesso seguro a recursos do hospedeiro.

- **Benefícios Frente a Soluções Tradicionais (Containers/JVM):**

- **Velocidade Quase Nativa:** Sobrecarga do bytecode WASM com JIT/AOT é de apenas 1,1x a 1.5x do código nativo compilado.
- **Footprint Mínimo:** Tamanho de imagem em KBs (vs Megabytes de Docker) e instanciamento em microssegundos (sem overhead de cold start).
- **Segurança Baseada em Capabilidades:** Bloqueio de rede e E/S por padrão, liberados explicitamente via flags do runtime (ex: WasmEdge).

- **Fundamentação Teórica (Ribeiro Junior et al., 2020):**
 - Concebeu a modelagem matemática e a lógica do filtro kNN com RLE aplicado a dados climáticos de tomateiros.
- **Avanços e Diferenciais Introduzidos neste PGC:**
 - 1 **Implementação Prática Real:** Migração do escopo analítico puro para um ecossistema distribuído completo em Rust e WASM rodando no runtime WasmEdge.
 - 2 **Validação Estatística Offline:** Desenvolvimento nativo de um gerador estocástico bivariado via estimador KDE e transformada de Box-Muller em Rust/WASM.
 - 3 **Emulação Multiambiente:** Configuração de proxies e servidores para avaliação do impacto físico em rede ao deslocar o processador entre as camadas (Thing, Fog e Cloud).

Arquitetura do Ecossistema Distribuído

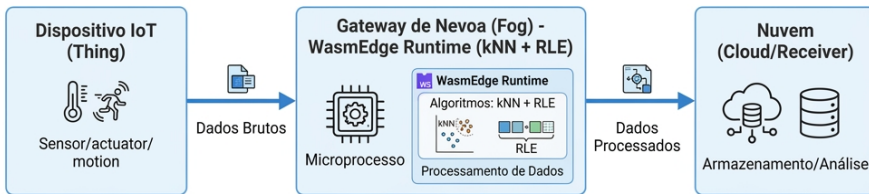


Figura: Arquitetura lógica em três fases: Thing, Edge/Fog e Cloud.

- **Filtragem via k-NN (k=3):**

- Normalização Min-Max em tempo real:

$$x'_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- Distância Euclidiana bidimensional para classificar status agrônômico do tomateiro em 11 classes (Tabela agrônômica baseada no ciclo da cultura).

- **Compressão Run Length Encoding (RLE):**

- Transforma strings sequenciais repetitivas de classes de caractere único em contadores e identificadores.
- Exemplo: O sinal contínuo 0000WWAA de 9 bytes é reduzido para 403W2A (6 bytes), economizando banda passante em transmissões por buffer temporal.

- **Linguagem Rust e Bibliotecas:**

- **Hyper / Reqwest:** Para comunicação HTTP cliente/servidor assíncrona baseada em sockets WASI compatíveis com o WasmEdge.
- **CSV / Serde:** Leitura de alta performance de dados brutos e serialização JSON estruturada ultraleve.

- **Sandbox WasmEdge e Segurança:**

- Mapeamento explícito de diretórios pela flag `-dir` . para dar permissão controlada de E/S de arquivos (leitura de datasets e persistência de relatórios).
- Ambiente isolado que impede ataques de estouro de buffer ou acesso irrestrito ao sistema operacional hospedeiro.

- **Base de Dados Física Real (entrada.csv):**
 - 44.023 registros climáticos bivariados coletados com taxa de amostragem de 5 segundos.
- **Buffers de Transmissão Avaliados:**
 - Cinco janelas de lotes temporais simuladas: **6** registros (30s), **60** (5min), **720** (1h), **4.320** (6h) e **20.000** (27,7h).
- **Cenário de Execução:**
 - Emulação executada em distribuição Ubuntu Linux via subsistema WSL em máquina pessoal equipada com Windows 11.

Objetivo do Gerador em Rust

Validar se o ecossistema e o classificador agrônômico se comportam de forma consistente sob dados estocásticos modelados a partir do ambiente climático original.

- **Ajuste Estatístico (Offline - KDE):** Ajuste da curva bivariada de densidade por estimador KDE gaussiano ($h = 0.5$) em Python.
 - *KDE (O "Cérebro"):* Mapeia o relevo de probabilidade e a correlação conjunta real do clima.
- **Amostragem em Rust/WASM (Box-Muller):** Geração de 40.000 pontos sintéticos utilizando a transformada Box-Muller:

$$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$$

- *Box-Muller (O "Motor"):* Transforma números planos (uniformes) em ruído normal (sino) ao redor do relevo.
- U_1, U_2 : Variáveis uniformes independentes em $(0, 1]$, geradas pelo WasmEdge.
- Z_0 : Ruído com distribuição normal padrão $N(0, 1)$ adicionado ao ponto base.

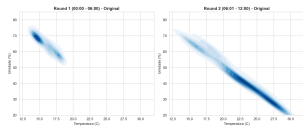
- **Teste de Kolmogorov-Smirnov (Duas Amostras):**

- Temperatura: estatística de 0,0081 e **p-valor de 0,1385**.
- Umidade: estatística de 0,0041 e **p-valor de 0,8974**.
- **p-valor (KS-Test):** *Mede a probabilidade de as amostras virem da mesma distribuição. Se $p > 0,05$ (limite crítico), provamos que os dados reais e sintéticos são estatisticamente indistinguíveis.*

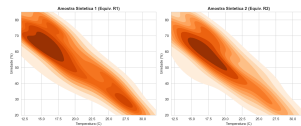
- **Correlação Linear (Pearson r):**

- Correlação Real: $-0,6429$.
- Correlação Sintética (Rust): $-0,6396$ (Desvio absoluto: 0,0033).
- **Pearson r :** Mede o quanto as variáveis caminham juntas. O valor negativo ($\approx -0,64$) reflete a física real do clima (temperatura sobe $\rightarrow$ umidade desce), perfeitamente preservada pelo gerador.

Análise Visual: Curvas de Densidade Conjunta



(a) Base Real (entrada.csv)

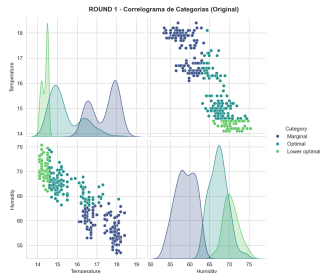


(b) Base Sintética (Rust/WASM)

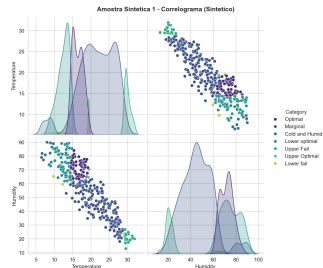
Figura: Comparativo das curvas de contorno de densidade de probabilidade conjunta.

- As metades sintéticas (Amostras 1 e 2) são idênticas entre si e cobrem a amplitude diagonal global. Isso ocorre porque o gerador KDE é um modelo probabilístico estático e *i.i.d.* (não modela a ordenação cronológica temporal da série física original).

Análise Visual: Correlogramas e Dispersão Cruzada



(a) Round 1 - Original



(b) Round 1 - Sintético

Figura: Correlogramas de Temperatura vs Umidade classificados por kNN (Round 1).

- Enquanto o Round 1 real concentra-se em 14 a 19°C (ativando 3 classes kNN), o sintético dispersa-se por todo o espectro global (5 a 30°C) e ativa classes adicionais, pois a amostragem aleatória do KDE reflete a densidade global conjunta.

Análise Visual: Densidade Global Completa

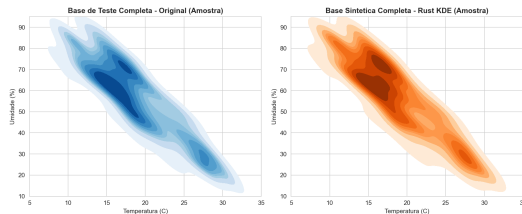


Figura: Comparação de Densidade Bidimensional Completa (Original vs Sintético).

- Sob a perspectiva global, o gerador KDE em Rust recupera a estrutura estocástica latente com altíssima fidelidade, com contornos e centros de adensamento muito próximos à base real.
- Contudo, a perda de ordenação temporal (modelo *i.i.d.*) introduz ruído de alta frequência na série sintética. Isso reduz as sequências de repetição e explica a menor eficiência do RLE nos dados simulados.

Resultados de Eficiência de Compressão

Tabela: Tamanho médio acumulado e redução de dados por tamanho de lote.

Tamanho Lote	Original (B)	kNN Puro (B)	kNN+RLE (B)	Redução kNN+RLE (%)
6	222	6	11,8	94,68%
60	2.222	60	75,7	96,59%
720	26.732	720	814,6	96,95%
4.320	160.392	4.320	4.908,4	96,93%
20.000	742.668	20.000	22.842,5	96,92%

* A redução máxima observada foi do **kNN Puro (estável em 97,84%)**, superando o kNN+RLE.

Discussão: Fenômeno de Expansão de Símbolo Único

O que explica a ineficiência do RLE?

A codificação RLE opera agrupando sequências de caracteres repetitivos (ex: 30 para três status Ótimo consecutivos). Contudo, devido à variabilidade natural e ruídos físicos de alta frequência dos sensores climáticos de campo, os caracteres alternam-se rapidamente no buffer.

* **Expansão de Símbolo Único:** Se não há repetições sequenciais de categorias e elas alternam a cada passo, o RLE precisa emitir um contador 1 para cada caractere (ex: a cadeia de 3 bytes `0 W A` torna-se `101W1A`, consumindo 6 bytes). * **Diretriz de Projeto:** Em alfabetos curtos sob ruído de sensores físicos, a compressão de entropia RLE gera sobrecarga de rede, sendo mais eficiente transmitir apenas a saída bruta rotulada de caractere único do kNN.

- **Viabilidade Tecnológica:**

- Rust e WebAssembly (WASM/WASI) provaram-se altamente eficientes e leves para estruturar a computação descentralizada em nós de névoa agrícolas.

- **Filtragem e Compressão:**

- O kNN atuou como um excelente pré-filtro semântico, reduzindo os dados climáticos reais em **97,84%** de volume de payload bruto de telemetria.
- A combinação com RLE mostrou-se contraprodutiva para buffers temporais menores devido ao ruído físico dos sensores.

- **Validação Estocástica:**

- O gerador bivariado em Rust/WASM replicou com sucesso e alta consistência estatística (p-valor de KS > 0.05) as propriedades de variabilidade do clima real.

❶ Orquestração Dinâmica de Tarefas (Workload Migration):

- Implementar mecanismos em WASM para migrar a lógica do processamento (kNN) do Thing para a Névoa de forma autônoma e dinâmica de acordo com a oscilação da banda LoRaWAN.

❷ Validação em Hardware Físico Embarcado:

- Avaliar a performance do WasmEdge rodando em nós microcontroladores de campo reais baseados em ESP32 ou Raspberry Pi Zero.

❸ Algoritmos de Compressão Adaptativos:

- Investigar a utilização de outros codificadores (ex: Huffman ou LZW) que lidam melhor com alfabetos curtos altamente flutuantes.

- Aos meus pais, pelo amor incondicional, apoio constante e incentivo.
- Aos meus amigos Leandro Diomar e Carlos Quintino, pela parceria científica, discussões técnicas e noites de estudo compartilhadas.
- Ao orientador Prof. Dr. Carlos Alberto Kamienski, pela condução, ensinamentos de pesquisa e paciência acadêmica.
- À Universidade Federal do ABC (UFABC), por fornecer ensino público e gratuito de altíssima qualidade científica.

Obrigado pela atenção!

Perguntas e considerações da banca avaliadora.